

CCLog: Actionable APT forensics via fused log semantics and provenance graph topology

Zhichao Hu^{a,} , Likun Liu^a, Hongjie Li^a, Chen Song^a, Mengmeng Ge^{a, d, *}, Qing Guo^b, Lina Ma^c, Xiangzhan Yu^a

^a School of Cyberspace Science, Harbin Institute of Technology, Harbin 150001, China

^b School of Electronics and Information Engineering, Harbin Institute of Technology, Harbin 150001, China

^c School of Management, Harbin Institute of Technology, Harbin 150001, China

^d College of Computing and Data Science, Nanyang Technological University, Singapore 639798, Singapore

ARTICLE INFO

Keywords:

APT
Provenance graph
Pretrained model
Attack provenance

ABSTRACT

Advanced Persistent Threats (APTs) are highly targeted, stealthy, and sophisticated cyber attacks that pose significant risks to network services, infrastructure, and data security. APT actors primarily target government agencies, research institutions, defense systems, and e-commerce platforms, creating growing cybersecurity challenges. Existing APT detection methods rely on rule-based models with predefined attack signatures or deep learning approaches that learn attack patterns from large-scale data. However, they suffer from high false positive rates, limited contextual understanding, and high computational overhead, making them ineffective against evolving threats. To overcome these limitations, we propose *CCLog*, a context-aware correlation analysis framework for APT detection and attack provenance. *CCLog* constructs a provenance graph from system logs, applies a BERT-based pre-trained model for log analysis, and employs a Variational Autoencoder for anomaly detection. Attack path reconstruction is optimized using a Steiner tree approach with shortest-path heuristics and a greedy algorithm, improving accuracy while reducing computational costs. Experimental results show that in the APT attack detection phase, *CCLog* achieves an average F1-score of 0.9012 across five datasets including CADETS, ARENA, THEIA, Trace, and clearscope, with an average improvement of 18.66% and a 4.23% improvement over the second-ranked method. In the APT attack provenance phase, it achieves an average F1-score of 0.8663, with an average improvement of 16.78% and a 0.59% improvement over the second-ranked method. Additionally, attack path reconstruction achieves optimal or near-optimal performance while reducing resource consumption by 30%. These findings highlight the effectiveness and practicality of *CCLog* for real-world APT detection and forensic analysis, advancing cybersecurity analytics.

1. Introduction

Advanced Persistent Threat (APT) refers to a highly targeted, stealthy, long-term, and technically sophisticated cyber attack that poses significant threats to network services, infrastructure security, data protection, and system integrity [1–4]. APT attack groups target a wide range of industries, including government agencies, research and education, national defense, and e-commerce. With the increasing volume and evolving sophistication of malicious activities, traditional security defenses—such as Intrusion Prevention Systems (IPS), Security Information and Event Management (SIEM) tools, and Web

Application Firewalls (WAF)—struggle to effectively detect and mitigate APT attacks. This presents unprecedented challenges to global cybersecurity [5–7].

In practice, APT detection faces two key challenges: (1) **High False Positive Rates and Alert Fatigue**: Intrusion detection systems generate excessive false positives, leading to analysis backlogs and alert fatigue due to the burden of verifying security alerts. Additionally, over 99% of logged events are benign, causing imbalance issues that increase false positive rates and reduce detection reliability. Furthermore, mimicry attacks are difficult to distinguish with traditional blacklist or signature-based detection. (2) **Long Attack Duration and Difficulty**

* Corresponding author at: School of Cyberspace Science, Harbin Institute of Technology, Harbin 150001, China.

E-mail addresses: huzhichao@hit.edu.cn (Z. Hu), liulikun@hit.edu.cn (L. Liu), 22S003118@stu.hit.edu.cn (H. Li), 2021111582@stu.hit.edu.cn (C. Song), gmm@hit.edu.cn (M. Ge), qguo@hit.edu.cn (Q. Guo), malina@hit.edu.cn (L. Ma), yxz@hit.edu.cn (X. Yu).

<https://doi.org/10.1016/j.comnet.2025.111660>

Received 10 May 2025; Received in revised form 23 August 2025; Accepted 25 August 2025

Available online 31 August 2025

1389-1286/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

in Attack Path Reconstruction: APT attacks unfold over extended periods, making it difficult to reconstruct complete attack chains from isolated events. Existing methods struggle to process long-term attack sequences, often capturing only low-level anomalies while failing to model complex multi-stage attack dependencies. This results in incomplete APT identification and inefficient attack path reconstruction.

APT attack detection primarily relies on either predefined rule-based models or deep learning approaches that learn and identify complex attack patterns from large-scale data [5,6,8–13]. While these methods have demonstrated promising detection performance, they still suffer from several key limitations: (1) Dependence on predefined attack patterns [9,13], making them ineffective against novel or evolving threats. (2) Limited contextual understanding [10,12], restricting their ability to capture long-term dependencies and attack correlations. (3) High computational overhead [11], making real-time or large-scale deployment impractical. As APT attack strategies continuously evolve, existing detection methods struggle to adapt to the dynamic threat landscape [14–16], necessitating more robust, adaptive, and efficient detection frameworks.

Therefore, we focus on in-depth log analysis, leveraging semantic modeling and contextual correlation to enhance actionable APT forensics. The core ideas and major contributions of our proposed approach, *CCLog*, are as follows.

- We propose a pre-trained model for log analysis, which performs deep semantic analysis on node attributes and inter-node operations while incorporating clustering of similar samples. This enhances the understanding of attack behaviors and provides a solid foundation for attack node detection.
- We propose a VAE-based (Variational Autoencoder, VAE) attack node detection method. It enhances traditional reconstruction loss-based anomaly detection by incorporating K-Means clustering to compute attack score. This approach effectively reduces false positive rates and improves the accuracy and reliability.
- We employ an optimized Steiner Tree algorithm for offline attack path reconstruction, achieving a balance between detection accuracy and resource efficiency.

2. Related works

2.1. APT attack detection

APT attack detection techniques can be categorized into two approaches: rule-based models and machine learning methods. The former identifies known malicious activities based on predefined signatures such as command sequences, network behaviors, and malware characteristics, while the latter leverages deep learning to recognize complex attack patterns from large-scale datasets [5,6,8]. Provenance graphs provide a structured approach to track system activities, including file operations, process executions, and network connections, making them effective for analyzing complex attack chains [9–13].

GNN-based (Graph Neural Network, GNN) methods have shown promise in capturing structural dependencies and modeling sophisticated attack behaviors [17]. For instance, Mask [6] utilizes Masked Graph Representation Learning to capture deep feature representations of normal system behavior. In contrast, THREATTRACE [18] applies the GraphSage algorithm for provenance graph embedding learning, aligning node embedding vectors for classification to identify anomalous patterns. APT-KGL [7] further improves detection performance by constructing a heterogeneous provenance graph (HPG) to model system entities and contextual relationships. Despite their advantages, GNN-based approaches still face limitations such as high computational overhead and limited interpretability [19,20]. To enhance efficiency, AIRTAG [21] introduces an unsupervised learning method that directly processes raw log text, reducing computational costs while accelerating detection. FLASH [22] improves efficiency via an embedding recycling

mechanism, but this mechanism relies heavily on the completeness of node attributes—struggling to function in scenarios with significant attribute missing, where real-time GNN embedding generation remains necessary. While its node-level detection offers better interpretability than graph-level methods, it remains insufficiently intuitive for ordinary security analysts.

While these methods have demonstrated effectiveness, they still struggle with prior knowledge dependence, semantic relationship modeling, and computational efficiency [14–16]. These challenges underscore the necessity for more adaptable and interpretable APT detection frameworks that can efficiently analyze large-scale security logs and evolving attack patterns.

Nowadays, APT detection based on large models is gradually becoming prevalent. With hundreds of millions of parameters for log analysis, large models demonstrate superior generalization ability and context understanding compared to GNNs, and introducing them into APT detection will be a future development direction. APT-LLM [23] incorporates various general pre-trained models into APT detection, featuring strong universality and time efficiency. In contrast, our approach involves fine-tuning specifically for APT scenarios, which results in stronger targeting but is more time-consuming. In terms of data processing, we decompose logs into process-level structured information such as cmdline and filepath, while APT-LLM converts logs into descriptive text, which is more suitable for LLM-based semantic modeling.

2.2. APT attack provenance

APT provenance methods focus on attack path reconstruction and APT group affiliation analysis. Recent approaches leveraging causal analysis, graph neural networks (GNNs), and deep learning have improved attack traceability and detection efficiency.

APT attacks involve multi-stage infiltration, making attack path reconstruction a key challenge. Provenance graph analysis plays a crucial role in this process [6,18,24,25]. Incorporating temporal analysis, Yin et al. [26] proposed a provenance graph-based method that constructs system behavior sequences and extracts temporal features for precise attack path tracking. To further address distributed issues, Liu et al. [27] introduced the TRACEGADGET framework, integrating federated provenance graphs with cross-host log analysis to comprehensively reconstruct APT attack paths and track lateral movement.

Beyond log data, network traffic analysis is also critical in APT provenance. Adnan et al. [28] proposed an unsupervised machine learning-based framework that analyzes HTTP and SMTP traffic to identify key attack nodes. Further more, Chen and Gong [29] enhanced accuracy by fusing ICS logs, network traffic, and device states for attack path analysis. In addition to attack path provenance, identifying APT attack groups is crucial for defense strategies. Expanding on this, Li et al. [30] introduced the T-Trace method, combining log correlation analysis with tensor decomposition to extract APT event communities and build precise provenance graphs, significantly enhancing attribution accuracy. Wang et al. [31] further improved interpretability by using shapelet-based segmentation to analyze temporal morphological features in APT malware execution paths.

These studies highlight advancements in event correlation, organization identification, and real-time attack path reconstruction. However, challenges remain, including limited dataset diversity, reliance on labeled data, and the lack of large-scale scenario validation. Future research should incorporate cross-domain datasets and unsupervised learning to improve model generalization and robustness.

Table 1
Datasets information.

Datasets	Tag	Size	Records	Source	Duration
CADETS (2019)	Benign	271 GB	11,494,277	DARPA	247 h
ARENA (2023)	Attack	38 GB	4,905,834		
THEIA (2019)	Benign	59 GB	12,493,189	Sangfor	336 h
Trace (2019)	Attack	12 GB	3,905,128		
clearscope (2019)	Benign	85 GB	3,501,561	DARPA	247 h
	Attack	4 GB	252,297		
	Benign	61 GB	2,389,233	DARPA	264 h
	Attack	3 GB	67,382		
	Benign	441 GB	4,199,309	DARPA	247 h
	Attack	432 GB	4,273,003		

3. Proposed method

3.1. Problem statement

This study focuses on APT attack detection and provenance analysis, addressing key challenges in existing methods, including high false positive rates, difficulty in reconstructing complex attack paths, and high resource consumption. To overcome these limitations, we adopt a provenance graph-based approach that detects APT-related entities from large-scale logs and reconstructs the corresponding attack paths.

For a given provenance graph $G = (V, E, \omega)$, where ω represents edge weights, the problem is divided into two stages: (1) **Attack Entities Detection**: Identify the set of attack nodes $T \in V$. (2) **Attack Path Reconstruction**: Using T as the target nodes, find a subgraph $G' = (V', E')$ that minimizes the total edge weight, as formulated in Eq. (1).

$$\min \sum_{e \in E'} \omega(e) \quad \text{s.t.} \quad T \in V' \quad (1)$$

3.2. Datasets

We utilize datasets from DARPA [32] and ARENA [33] for research. The DARPA includes CADETS, THEIA, Trace, and clearscope. Their detailed information are presented in Table 1.

1. The DARPA, collected from a host network during the two-week third adversarial engagement of the DARPA Transparent Computing program, were utilized for this research. This engagement involved multiple teams tasked with collecting audit data from diverse platforms, executing attacks throughout the engagement period, and conducting data analysis for attack detection and forensic investigation. The red team responsible for launching attacks also generated benign background activities, thereby enabling the modeling of normal system behaviors. (1) **CADETS** dataset was captured via the Causal, Adaptive, Distributed, and Efficient Tracing System (CADETS) on FreeBSD. (2) **THEIA** is a system for tagging and tracking multi-level host events; it instruments Ubuntu Linux machines during the engagement. (3) **TRACE** captures detailed audit records of APT attacks and concurrent benign activities in the enterprise network environment of the DARPA TC program to form its public APT attack dataset. (4) **clearscope** instruments the entire Android mobile software stack to capture the provenance of operations on mobile devices.
2. The **ARENA**, published by Sangfor in 2023, emulates a production environment over a 14-day period, encompassing logs from both Windows and Ubuntu systems. It deploys software such as Apache, Nginx, and PostgreSQL, and conducts simulated APT attacks, including APT29, APT32, and exploitation of the Log4j2 vulnerability (CVE-2021-44228), with detailed attack steps documented.

Both datasets provide comprehensive system activity and attack behavior data, making them suitable for security analysis and provenance research.

Table 2
Operation definition.

Operation category	Operation	Build graph
FILE_OP	read write rename mmap	operation process → file (identified by its path or descriptor)
PROCESS_OP	fork vfork pipe kill	parent process → child process
NET_OP	sendto connect bind	operation process → network resource (destination network address and port)
PERM_OP	setuid seteuid setlogin setgid	process or file → user

Table 3
The size of the provenance graph.

Datasets	Entities (Graph nodes)	Relations (Graph edges)
CADETS	20,652,968	144,319,564
ARENA	8,899,725	122,473,869
THEIA	11,889,821	228,824,718
Trace	2,416,007	10,978,024
clearscope	9,485,265	40,842,345

3.3. Overview

This paper proposes *CCLog*, an actionable method for APT detection based on log semantic fusion and provenance graph analysis, as illustrated in Fig. 1.

It begins by constructing a provenance graph through the extraction of dependency relationships from logs. Subsequently, the BERT is employed to capture contextual semantic information from log data, enabling the identification of attack features and the detection of malicious entities based on VAE. Finally, a graph optimization algorithm is applied to reconstruct the APT attack path. This integrated methodology facilitates comprehensive detection and provenance analysis of APT attack behaviors.

3.4. Provenance graph build

In provenance graph, nodes represent system entities (e.g., processes, files, network connections), while edges denote interaction relationships (e.g., read/write, connect) between entities, along with timestamp information. We divided system logs into four types, as defined in Table 2. A typical provenance graph, as shown in Fig. 2, clearly illustrates the flow of attack events, providing a structured representation of system activities for effective attack detection and analysis.

The size of the provenance graphs constructed from the 5 datasets is summarized in Table 3.

3.5. APT attack entities detection

To accurately identify entities directly related to APT attacks within the initial provenance graph, we use the BERT model to extract entity attributes and contextual semantic information. Concurrently, VAE is utilized to learn the feature distribution of normal behaviors based on contextually associated features. This dual approach enables effective detection of attack nodes that deviate from normal patterns, enhancing the precision of APT attack identification.

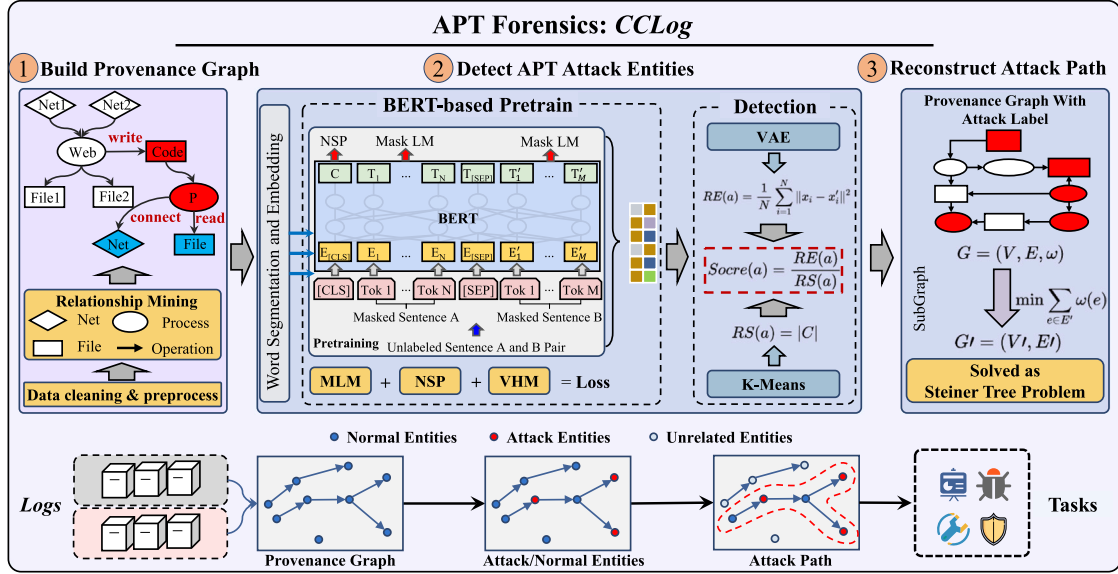


Fig. 1. Framework Overview of *CCLog*. There three main steps: (1) **Build Provenance Graph**: Through log preprocessing, we extracts dependency relationships from system logs for Net, Process, and File operations, modeling the data as a provenance graph. The graph accurately captures entity interactions and causal relationships, preserving APT attack behaviors. (2) **Detect APT Attack Entities**: To identify attack-related entities, *CCLog* integrates a BERT model and VAE: BERT extracts semantic contextual features from entity identifiers, enhancing deep semantic representation and context modeling. VAE reconstructs feature vectors and leverages reconstruction errors along with redundancy scoring to identify attack entities. (3) **Reconstruct Attack Path**: *CCLog* use optimized Steiner tree algorithm to reconstruct attack path by generating a minimum-cost subgraph. To enhance path continuity and coherence, non-attack entities can be introduced when necessary. The framework iteratively refines the optimal attack path, removes redundant entities and operations, and ensures completeness and accuracy of the reconstructed attack sequence.

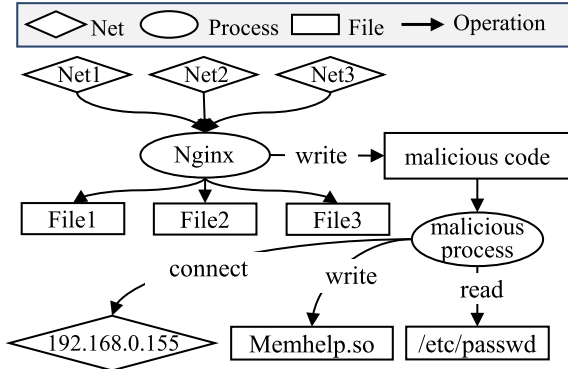


Fig. 2. Example of Provenance Graph. The graph illustrates an attack where an attacker exploits a legitimate service (e.g., *Nginx*) to inject malicious code, triggering a malicious process. This process accesses critical system files (e.g., */etc/passwd*), modifies dynamic libraries (e.g., *Memhelp.so*), and establishes an external connection (e.g., 192.168.0.155) for data exfiltration.

3.5.1. BERT-based pretrain for log analysis

During the pre-training phase of *CCLog*, two key aspects are emphasized: (1) capturing node semantics and node operation semantics to fully understand contextual relationships, and (2) ensuring the model comprehensively learns the patterns of normal system operations. By integrating these objectives, the model gains a robust understanding of typical system behavior, enabling it to effectively identify deviations indicative of attack activities.

For pre-training input, the BERT Tokenizer's WordPiece tokenizer is first employed to segment node identifiers into tokens. For example, the node identifier **grep processor proc cpuinfo** is tokenized into **[grep, processor, proc, cpu, ##info]**. This tokenization approach decomposes unknown or rare words into smaller, known sub-units, enhancing the model's ability to handle unseen vocabulary. Subsequently, the

tokenized input is transformed into the input format required by the BERT model, aligning with the specific pre-training task objectives. This process ensures that the model can effectively process and learn from diverse and complex log data.

We design 3 pre-train tasks: Masked Language Modeling (MLM), Next Sentence Prediction (NSP), and Volume of Hypersphere Minimization (VHM).

(1) **Pretrain Task 1 (MLM)**: The MLM enhances language understanding by predicting randomly masked words within the input, enabling the model to learn the rich semantic information embedded in node attributes. As shown in Fig. 3, this process helps the model capture meaningful representations of individual nodes.

It measures the model's accuracy by whether predicting the right masked words. The model computes the cross-entropy loss based on the probability distribution of the predicted and ground-truth words, as formulated in Eq. (2).

$$L_{MLM} = -\frac{1}{N} \sum_{i=1}^N \log P(\varpi_i | C_i), \quad (2)$$

where N represents the number of masked words, ϖ_i denotes the i th masked word, and C_i refers to the context of the i th masked word. The term $P(\varpi_i | C_i)$ represents the probability distribution predicted by the model for the masked word based on its context.

(2) **Pretrain Task 2 (NSP)**: The NSP further enhances the model's contextual understanding by determining whether two nodes are logically coherent. As shown in Fig. 4, this task enables the model to learn structural and semantic relationships between nodes, improving its ability to capture contextual dependencies.

Specifically, for each input node pair, the model outputs a binary probability distribution, indicating the likelihood that the pair is connect or not. The model then computes the cross-entropy loss based on the ground-truth labels and the predicted probability distribution, as formulated in Eq. (3).

$$L_{NSP} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]. \quad (3)$$

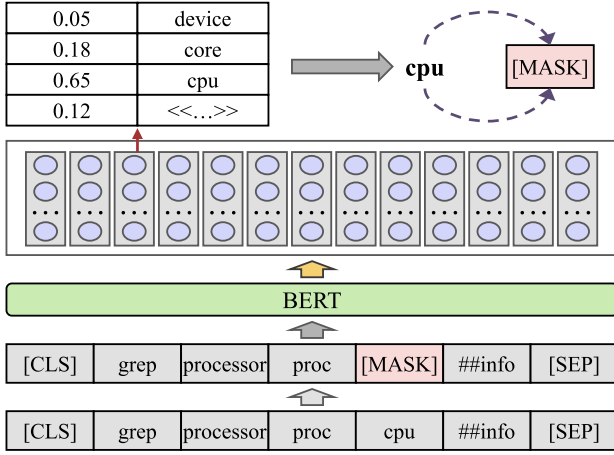


Fig. 3. MLM Illustration. During input processing, a randomly chosen portion of words (e.g., “cpu”) is masked. The BERT model then generates a probability distribution over possible words for the [MASK] position. As shown in the figure, the model assigns the highest probability (0.65) to “cpu”, making it the predicted word to replace [MASK]. This task enables the model to learn semantic representations of node attributes, enhancing its understanding of contextual dependencies within log data.

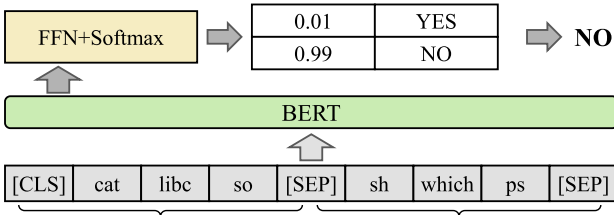


Fig. 4. NSP Illustration. The model determines whether the first node has an edge pointing to the second. By this way, it enables the model to understand the contextual relationships between node representations.

Here, N represents the number of node pairs, y_i denotes the ground-truth label for the i th node pair (1 for connect, otherwise 0), and $\hat{y}_i$ is the predicted probability for the i th node pair.

(3) Pretrain Task 3 (VHM): The VHM enables the model to cluster normal data points while pushing attack data away from the center, enhancing the separation between normal and malicious behaviors. As shown in Fig. 5, this task improves the model’s ability to distinguish between normal and attack nodes in the feature space.

The objective function of the VHM is designed to minimize the squared Euclidean distance between each input node’s feature vector h_j and the center vector c , denoted as $\frac{1}{N} \sum_{j=1}^N \|h_j - c\|^2$. The loss function is formally defined in Eq. (4). By minimizing L_{VHM} , the BERT model clusters normal node representations in the feature space while pushing attack feature vectors away from the center, enhancing the model’s ability to distinguish normal and malicious data.

$$L_{VHM} = \frac{1}{N} \sum_{j=1}^N \|h_j - c\|^2. \quad (4)$$

In summary, the total pre-training loss consists of the MLM loss, NSP loss, and VHM loss, as formulated in Eq. (5).

$$L_{Total} = L_{MLM} + \lambda L_{NSP} + \alpha L_{VHM}. \quad (5)$$

This combined loss ensures that the model effectively learns semantic understanding (MLM), contextual coherence (NSP), and feature space separation (VHM), enhancing its ability to distinguish between normal and attack nodes.

3.5.2. VAE-based anomaly model for attack node detection

CCLog employs a VAE-based anomaly detection model for attack node detection. During training, the VAE learns to model the representations of normal nodes and reconstruct them. The optimization process involves minimizing both the reconstruction loss and the KL loss.

During detection, anomalies are identified by evaluating the reconstruction loss. Commonly, the attack score is directly derived from the reconstruction loss, formulated as:

$$\frac{1}{N} \sum_{i=1}^N \|x_i - x'_i\|^2. \quad (6)$$

However, relying solely on reconstruction loss as the attack score lacks consideration for historical behavior consistency. This approach does not account for scenarios where a node’s behavior exhibits high consistency within the dataset. For instance, on a particular day, a system administrator may perform system maintenance or software updates, including patch installations and service restarts. These actions may deviate from the normal distribution, but they are legitimate activities rather than attacks. If only reconstruction loss is used for attack scoring, it may result in a high false positive rate, leading to excessive alerts and warnings.

To enhance detection and reduce false positives, we adopt a combined approach that integrates reconstruction loss and redundancy scoring when computing the attack score. The underlying principle is: (1) If an entity type exhibits high diversity, meaning it has multiple behavioral clusters, deviations in its behavior should have a lower anomaly score, as variations in behavior are expected. (2) If an entity type has only a few clusters, indicating consistent behavior, any deviation from its typical feature distribution is more likely to be an anomaly.

To quantify redundancy scoring ($RS(a)$), we apply the K-Means to cluster feature vectors of entities with the same identifier. The redundancy score is defined as the number of clusters (C) for the entity type of node a , formulated as: $RS(a) = |C|$, where C represents the number of clusters that the feature vectors of node a ’s identifier belong to within the dataset. By integrating reconstruction loss and redundancy scoring, we define the final attack score in Eq. (7).

$$Score(a) = \frac{RE(a)}{RS(a)}. \quad (7)$$

For a given node a , if $Score(a)$ exceeds a predefined threshold, it is classified as an attack node. Through this approach, the VAE effectively leverages BERT’s deep semantic representations to capture deviations in node behavior from normal patterns. By integrating reconstruction error and redundancy scoring, this method significantly reduces false positive rates and enhances the accuracy of attack node detection.

3.6. Attack path reconstruction

For provenance graph analysis and attack path reconstruction, we employ the Steiner Tree algorithm. This approach allows the inclusion of non-attack nodes, significantly enhancing path flexibility and optimizing connectivity between attack nodes. Compared to the Minimum Spanning Tree (MST) algorithm, the Steiner Tree algorithm not only connects all attack nodes but also selects additional intermediate nodes strategically, thereby reducing the total path weight. However, solving the Steiner Tree problem in large-scale provenance graphs is computationally expensive due to its high complexity. To address this, we adopt a hybrid approach combining the Shortest Path Heuristic Algorithm and a Greedy Algorithm. This method balances computational efficiency and path optimization, ensuring an accurate and scalable provenance analysis, as shown in Algorithm 1.

In this algorithm, the *findShortestPath* function uses a greedy strategy to find the shortest path between node pairs. It prioritizes neighboring nodes with higher attack scores ($Score$) as the next hop, aiming to capture the most attack-relevant events. Each iteration picks the lowest-cost path to connect two nodes, gradually building the attack

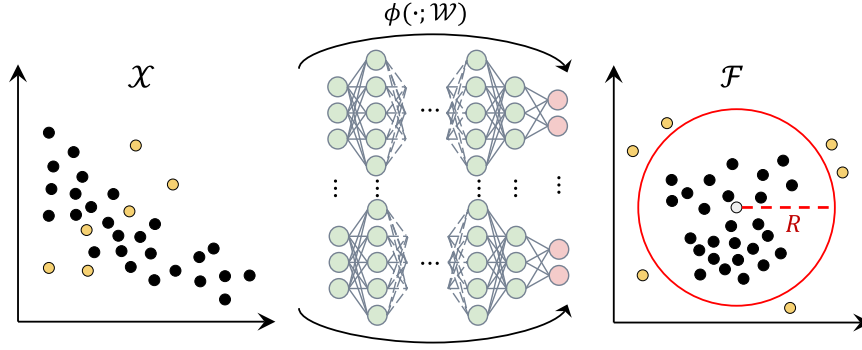


Fig. 5. VHM Illustration. $\phi(\cdot; W)$ represents the feature extraction process performed by the BERT model, where W denotes the model parameters. The visualization demonstrates how the VHM task clusters normal nodes while pushing attack feature vectors away from the center in the feature space.

Algorithm 1 Steiner Tree based attack path reconstruction.

Input: Provenance graph G , Attack node set V_t

Output: Attack path set S

```

1: Initialize the set of processed attack nodes  $V_p$  and the edge set  $S$  as empty.
2: for  $t_i \in V_t$  do
3:    $path \leftarrow []$ 
4:   if not  $Empty(V_p)$  then
5:     for  $t_j \in V_p$  do
6:        $t_{path} \leftarrow findShortestPath(G, t_i, t_j)$ 
7:        $path.add(t_{path})$ 
8:     end for
9:   else
10:     $V_p.add(t_i)$ 
11:   end if
12:    $S_i \leftarrow selectMinimumCostPath(path)$ 
13:    $S.add(S_i)$ 
14: end for
15: return  $S$ 

```

scenario. This strategy does not ensure a globally optimal solution but offers an effective approximation. To minimize the total weight of the Steiner Tree, the final edge set S may contain edges with non-attack nodes. Although these nodes are not in the attack node set, they are crucial for connecting attack nodes, constructing a low-cost Steiner Tree, and enhancing the coherence and efficiency of the reconstructed attack path.

The proposed method has two advantages. It extracts the minimum subgraph connecting attack related nodes in large scale provenance graph, cutting noise and irrelevant elements to boost accuracy and efficiency. It also employs a greedy algorithm to choose a node subset instead of full graph search, slashing complexity and overhead. It filters out irrelevant operations, making the attack path concise and interpretable for easy attacker tracing in forensics and threat probes.

4. Experimental results

In this section, we use the results on the real datasets to answer these 7 questions.

- RQ1: How effective is *CCLog* in attack entities detection? Is it competitive to other methods?
- RQ2: How effective is *CCLog* in attack path reconstruction? Is it competitive to other methods?
- RQ3: How robust is *CCLog* under adversarial attacks?
- RQ4: Does the pretrain model of *CCLog* works well for log analysis? How does the designed tasks contribute to the performance of *CCLog*?

- RQ5: How about the consumption of *CCLog*? Does it reduce the cost of time, memory, and cpu?
- RQ6: What are the key components and params that affect *CCLog*?
- RQ7: How does *CCLog* perform in practical detection?

4.1. Setup

Environment: All experiments were conducted on a server equipped with an Intel i9-13900KS CPU, 64 GB of RAM, and two NVIDIA RTX 4090 GPUs. The software was configured on Ubuntu 22.04 LTS, Python 3.9, and PyTorch 2.4.0.

Datasets: We utilized 90% of the normal nodes from the DARPA datasets (including CADETS, THEIA, Trace, and ClearScope) and the ARENA dataset for model training, with 18.49 million normal nodes from CADETS, 7.97 million from ARENA, 10.35 million from THEIA, 2.07 million from Trace, and 4.31 million from ClearScope. The test dataset was constructed using attack nodes and the remaining 10% of normal nodes.

Model: We select BERT-base architecture to build pre-trained model, which consists of 12 encoder layers and 12 attention heads. The hidden layer dimension is 768, while the feed-forward network (FFN) hidden layer dimension is 3072. Additionally, a dropout rate of 0.1 is applied. In total, the model comprises approximately 110 million parameters.

4.2. RQ1: The effectiveness of attack entities detection

To validate the effectiveness of the proposed method, we compare it with 6 commonly used attack detection approaches: DeepLog [10], cnnLog [12], ThreaTrace [18], LogBERT [11], Kairos [34] and Flash [22].

In this study, the threshold is set at the 90th percentile of the attack score distribution, where data points exceeding this threshold are classified as attacks. The attack score distribution obtained from training, taking CADETS and ARENA as examples, is shown in Fig. 6. The anomaly scores exhibit strong discriminative capability.

Table 4 presents a comprehensive comparison of experimental results across multiple datasets. Among these, *CCLog* demonstrates consistently superior or highly competitive performance over state-of-the-art baselines. Notably, on the ARENA dataset, it achieves top-tier results with an accuracy of 0.9655, recall of 0.9792, F1-score of 0.9669, and a notably low false positive rate (FPR) of 0.005. These outcomes underscore its effectiveness in accurately identifying advanced persistent threats while minimizing both false positives and false negatives. Furthermore, *CCLog* attains a Prec@5 of 0.8 and an NDCG of 0.9825, reflecting its robustness under data imbalance. On other datasets, it remains highly competitive, with an average F1-score of 0.9012 across all environments—an improvement of 18.66% over the baseline average and 4.23% over the second-best method. These results confirm

Table 4
Performance compasion of attack entities detection.

Dataset	Methods	Prec	Recall	F1	Prec@k	AUC-PR	NDCG	AUC	FPR (%)
clearscope	DeepLog	0.3090	0.4599	0.3696	0.2(k = 5)	0.4877	0.3823	0.4895	12.1
	cnnLog	0.5212	0.5542	0.5372	0.4(k = 5)	0.5501	0.5423	0.7999	7.8
	ThreaTrace	0.8801	0.8221	0.8501	0.7(k = 5)	0.8996	0.8278	0.9441	3.2
	LogBERT	0.5622	0.5897	0.5755	0.4(k = 5)	0.5902	0.6148	0.8113	9.4
	Kairos	0.7142	0.7999	0.7546	0.8(k = 5)	0.8780	0.8518	0.8964	1.8
	Flash	0.8346	0.7619	0.7966	0.8(k = 5)	0.8237	0.8983	0.8744	2.5
	<i>CCLog (ours)</i>	0.8333	0.8131	0.8232	0.8(k = 5)	0.8631	0.9347	0.9015	1.2
THEIA	DeepLog	0.3300	0.5400	0.4117	0.2(k = 5)	0.4312	0.3192	0.7120	4.3
	cnnLog	0.7681	0.5500	0.6404	0.6(k = 5)	0.5981	0.5541	0.7999	1.8
	ThreaTrace	0.8520	0.7860	0.8190	0.7(k = 5)	0.8760	0.7120	0.8340	4.8
	LogBERT	0.7683	0.5406	0.6347	0.6(k = 5)	0.6063	0.5904	0.8984	1.7
	Kairos	0.2759	0.8888	0.4210	0.4(k = 5)	0.5999	0.6542	0.8788	0.8
	Flash	0.8976	0.9135	0.9056	0.8(k = 5)	0.8374	0.7663	0.9307	0.6
	<i>CCLog (ours)</i>	0.8864	0.9228	0.9042	0.8(k = 5)	0.8174	0.7975	0.9228	0.6
ARENA	DeepLog	0.5593	0.5352	0.5469	0.2(k = 5)	0.4213	0.5871	0.6339	9.8
	cnnLog	0.9118	0.9064	0.9091	0.7(k = 5)	0.7432	0.7999	0.8523	0.8
	ThreaTrace	0.8973	0.8546	0.8764	0.8(k = 5)	0.8427	0.8619	0.9533	0.9
	LogBERT	0.7849	0.8014	0.7931	0.6(k = 5)	0.6133	0.6201	0.7928	4.3
	Kairos	0.8901	0.8576	0.8771	0.6(k = 5)	0.9101	0.8975	0.9661	2.8
	Flash	0.7862	0.7427	0.7651	0.6(k = 5)	0.7942	0.6488	0.7093	5.1
	<i>CCLog (ours)</i>	0.9549	0.9792	0.9669	0.8(k = 5)	0.9519	0.8818	0.9825	0.5
CADETS	DeepLog	0.5589	0.5468	0.5522	0.4(k = 5)	0.6129	0.4549	0.6874	4.4
	cnnLog	0.7325	0.7173	0.7248	0.7(k = 5)	0.7321	0.5989	0.7689	1.4
	ThreaTrace	0.8270	0.7540	0.7890	0.7(k = 5)	0.8420	0.8830	0.9050	0.2
	LogBERT	0.8218	0.8005	0.8110	0.8(k = 5)	0.7740	0.6293	0.8116	1.5
	Kairos	0.8000	0.9750	0.8880	0.8(k = 5)	0.8900	0.8208	0.8371	0.6
	Flash	0.9328	0.9471	0.9398	1.0(k = 5)	0.9341	1.0000	0.9392	0.5
	<i>CCLog (ours)</i>	0.8789	0.9279	0.9028	1.0(k = 5)	0.8374	1.0000	0.8993	0.9
TRACE	DeepLog	0.3347	0.4136	0.3698	0.0(k = 5)	0.3923	0.0000	0.4251	15.2
	cnnLog	0.7836	0.7923	0.7880	0.6(k = 5)	0.7588	0.6989	0.6689	2.8
	ThreaTrace	0.7200	0.7720	0.7450	0.8(k = 5)	0.7614	0.8174	0.8544	1.1
	LogBERT	0.5774	0.6043	0.5906	0.4(k = 5)	0.5319	0.4775	0.5940	7.4
	Kairos	0.9100	0.8321	0.8711	0.8(k = 5)	0.9212	0.8501	0.8721	1.5
	Flash	0.8547	0.9231	0.8876	0.8(k = 5)	0.8673	0.8716	0.9063	0.5
	<i>CCLog (ours)</i>	0.8991	0.9193	0.9091	1.0(k = 5)	0.8897	1.0000	0.8763	0.7

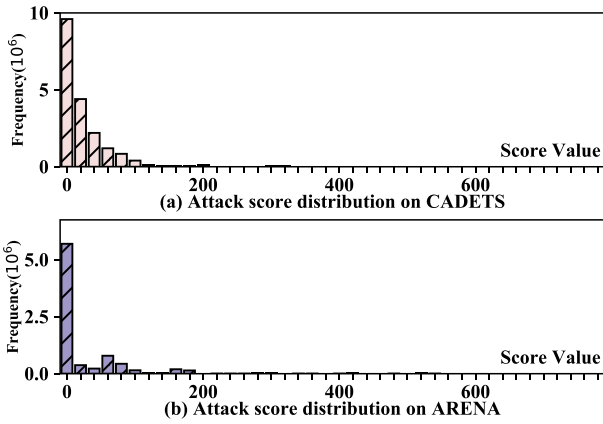


Fig. 6. The distribution of attack score over CADETS and ARENA. According to the 90th percentile, the threshold is set to 23.238 for the ARENA dataset and 35.487 for the CADETS dataset.

the generalizability and reliability of *CCLog* in diverse APT detection scenarios.

Complementing these findings, Fig. 7 illustrates the detection performance of *CCLog* in terms of F1-Score and NDCG@5. As shown in Fig. 7(a), *CCLog* achieves the highest F1-Score on most datasets and ranks second on ClearScope and CADETS, demonstrating consistent detection capability across heterogeneous log environments. In Fig. 7(b), it secures the top NDCG@5 value on four out of all datasets, further validating its strength in handling imbalanced data distributions. These

results collectively emphasize the adaptability of *CCLog* to varied log architectures and extreme attack scenarios.

In contrast, other methods exhibit certain limitations. DeepLog and cnnLog, due to their local feature extraction mechanisms, lack effective context modeling, resulting in lower recall across most datasets. Although LogBERT shows improved feature representation capability and achieves an accuracy of 0.8074, it suffers from high computational costs and suboptimal scalability on large-scale log data. THREATTRACE and KAIROS, constrained by their reliance on 2-hop neighborhoods and fixed time windows respectively, show reduced effectiveness in detecting long-span attacks. Flash, which uses Word2Vec-based encoding, is highly dependent on rich node attributes; its performance declines significantly when log semantics are sparse.

A detailed per-dataset analysis reveals that *CCLog* performs close to the best on clearscope, THEIA, and CADETS. Its use of a pre-trained BERT model enables deep semantic understanding of node attributes, while the VAE module computes anomaly scores through reconstruction error and behavioral redundancy. However, the method's strong dependency on command semantics impedes its ability to distinguish highly similar malicious and normal commands in clearscope. Additionally, underutilization of temporal features in THEIA limits its effectiveness against attacks of different durations and phases. In CADETS, missing log attributes degrade input quality for BERT, which in affects the VAE's detection performance.

Meanwhile, FLASH effectively integrates Word2Vec and temporal encodings to capture both semantic and sequential characteristics. Augmented with GNN-based context modeling and optimized graph traversal, it compensates for missing attributes and performs strongly on THEIA and CADETS. THREATTRACE employs GNNs to identify structural deviations between malicious and benign nodes without relying

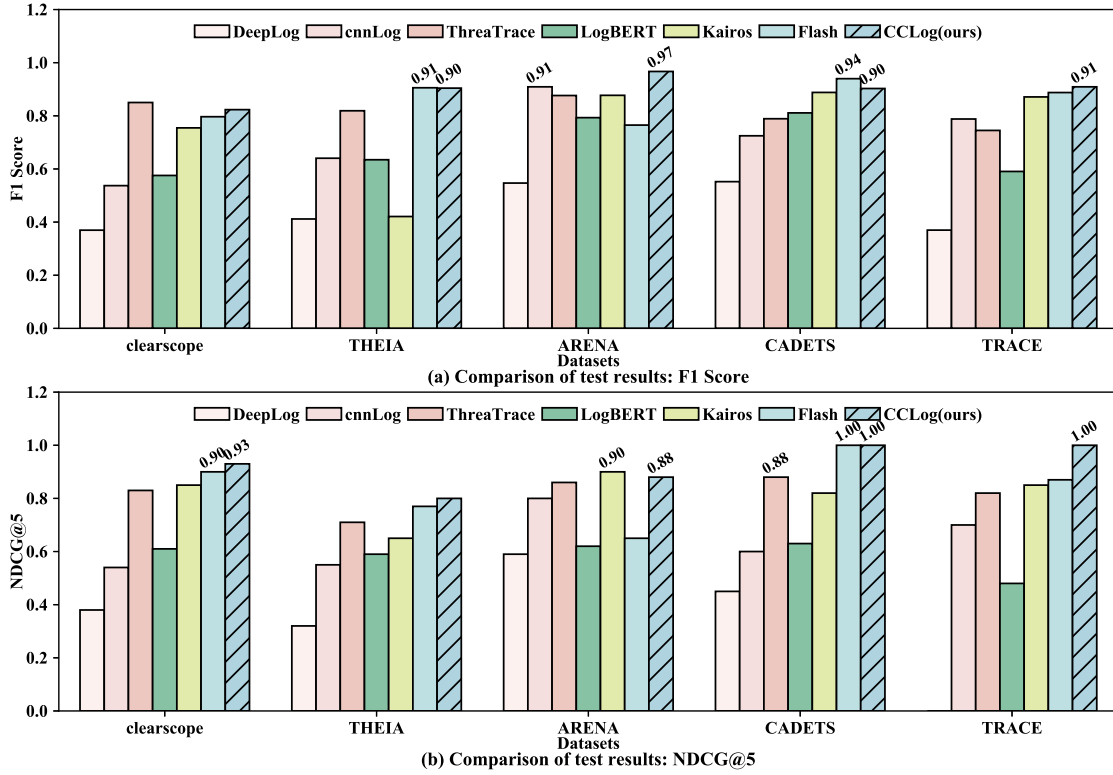


Fig. 7. Anomaly entities detection comparison on (a) F1-Score and (b) NDCG@5. By comparing CCLog with different methods on various datasets, it is demonstrated that CCLog has a better detection effect than other methods when data is imbalanced.

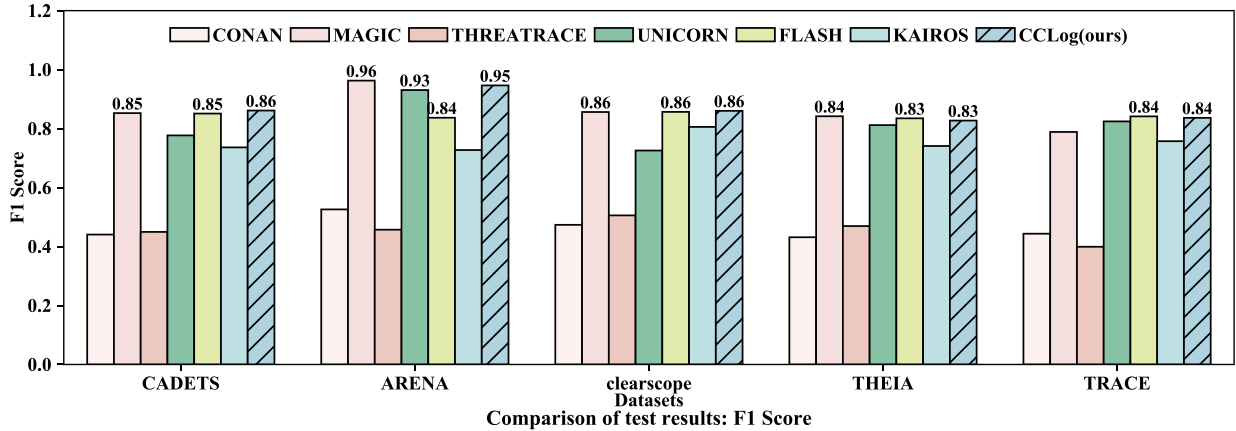


Fig. 8. Attack path reconstruct comparison. We compare the F1-Score of different methods across various datasets. CCLog achieves the highest F1 Scores on the CADETS and clearscope datasets, respectively, and ranks among the top on other datasets.

on command semantics, making it particularly effective in detecting legitimate tool abuse—as evidenced by its strong performance in clearscope.

4.3. RQ2: The effectiveness of attack path reconstruction

In this study, we compare attack path reconstruction performance with 6 existing methods: CONAN [24], MAGIC [6], THREATTRACE [18], UNICORN [25], Flash [22], and Kairos [34]. The experimental results are shown as Table 5 and Fig. 8.

CCLog achieves the highest F1-scores on both the CADETS (0.8612) and ClearScope (0.8602) datasets, while also ranking among the top performers across other datasets. With an average F1-score of 0.8663 over all datasets, it demonstrates an average improvement of 16.78%

over baseline methods and a 0.59% gain over the second-best approach. These results confirm the robustness and strong provenance capability of CCLog across diverse environments.

CONAN, due to its reliance on predefined state transition rules that lack adaptability to real data, performs the worst among all methods. Both CONAN and THREATTRACE exhibit consistently low recall across datasets, indicating limited coverage of attack paths. Their inability to capture contextual features—attributed to high computational complexity and incomplete feature extraction—further restricts their effectiveness against stealthy attacks and internal file-initiated attack steps.

MAGIC excels on THEIA and ARENA by capturing deep system features through masked learning. However, its performance declines on datasets with simpler attacks or smaller data volumes, suggesting

Table 5
Performance comparison of attack path reconstruction.

Dataset	Method	Accuracy	Precision	Recall	F1
CADETS	CONAN	0.3924	0.4189	0.4645	0.4406
	MAGIC	0.8576	0.9098	0.8021	0.8525
	THREATTRACE	0.3175	0.3845	0.5389	0.4496
	UNICORN	0.7297	0.6718	0.9213	0.7769
	FLASH	0.8711	0.9252	0.7947	0.8510
	KAIROS	0.7908	0.8571	0.7093	0.7362
	<i>CCLog (ours)</i>	0.8634	0.9170	0.8120	0.8612
ARENA	CONAN	0.4217	0.4683	0.5971	0.5259
	MAGIC	0.9553	0.9845	0.9379	0.9627
	THREATTRACE	0.5988	0.8425	0.3176	0.4571
	UNICORN	0.9273	0.9580	0.9045	0.9307
	FLASH	0.8296	0.8834	0.7947	0.8368
	KAIROS	0.7268	0.7693	0.6895	0.7272
	<i>CCLog (ours)</i>	0.9385	0.9016	0.9914	0.9464
clearscope	CONAN	0.3617	0.4692	0.4413	0.4736
	MAGIC	0.8753	0.9174	0.8027	0.8562
	THREATTRACE	0.3406	0.4333	0.6067	0.5056
	UNICORN	0.7705	0.6287	0.8576	0.7256
	FLASH	0.8972	0.8890	0.8244	0.8566
	KAIROS	0.7613	0.8914	0.7347	0.8057
	<i>CCLog (ours)</i>	0.8379	0.9537	0.7835	0.8602
THEIA	CONAN	0.4513	0.3770	0.4877	0.4315
	MAGIC	0.8061	0.9626	0.7479	0.8418
	THREATTRACE	0.2819	0.4274	0.5206	0.4694
	UNICORN	0.8281	0.7155	0.9382	0.8118
	FLASH	0.8406	0.9025	0.7764	0.8348
	KAIROS	0.7538	0.8237	0.6731	0.7410
	<i>CCLog (ours)</i>	0.8280	0.8913	0.7694	0.8272
TRACE	CONAN	0.3453	0.3777	0.4854	0.4435
	MAGIC	0.9262	0.8473	0.7379	0.7886
	THREATTRACE	0.3887	0.3151	0.5444	0.3992
	UNICORN	0.8541	0.7597	0.9011	0.8244
	FLASH	0.8972	0.8984	0.7911	0.8414
	KAIROS	0.8046	0.8317	0.6951	0.7572
	<i>CCLog (ours)</i>	0.8044	0.9343	0.7571	0.8364

limited adaptability. UNICORN offers stable and competitive performance through behavior pattern matching and real-time provenance analysis, but at the cost of high computational and memory overhead, especially with large-scale data.

KAIROS uses fixed time windows to segment long-term attacks, but often misclassifies rare benign edges as anomalous. This leads to mediocre performance on most datasets, with the exception of ClearScope. Flash's AEG discards nodes deemed non-alert, which causes loss of benign-yet-critical nodes within attack paths. Consequently, it achieves good but not optimal performance across most scenarios.

In contrast, *CCLog* leverages rich node attributes—such as entity types and contextual semantics—without manual rule definition or sole reliance on structural adjacency. This significantly improves both detection accuracy and adaptability to complex attack behaviors.

It is worth noting that the improvements of *CCLog* in provenance reconstruction over the second-best method are sometimes marginal (less than 1%). The method uses Steiner trees to optimize attack node linking within a host and improves path accuracy through semantic parsing and redundant edge pruning. However, it shows limitations in cross-host lateral movement detection, affecting performance in environments like THEIA and ARENA. Additionally, its static edge weighting strategy struggles with multi-branch attacks, as seen in TRACE.

FLASH, employing causal compression to filter irrelevant edges in real time, handles multi-branch attack chains more effectively. MAGIC, which uses a graph masked autoencoder to model cross-system dependencies, outperforms *CCLog* in cross-host scenarios.

On CADETS and clearscope, where attack paths are less complex, all methods perform near saturation. Still, *CCLog* maintains a slight edge owing to its Steiner tree-based redundancy reduction.

Table 6
Performance of *CCLog* under adversarial attack.

Attack	Datasets	Methods	F1	AUC-PR	FPR (%)
FGSM	CADETS	ThreaTrace	0.3842	0.3946	3.8
		cnnLog	0.3893	0.4092	4.1
		Kairos	0.3996	0.4315	4.5
		flash	0.4653	0.4783	3.6
		<i>CCLog</i>	0.5072	0.5127	3.2
	ARENA	ThreaTrace	0.6116	0.4736	2.9
		cnnLog	0.5001	0.5125	3.1
		Kairos	0.6567	0.6853	2.6
		flash	0.6911	0.6473	2.3
		<i>CCLog</i>	0.7033	0.6281	2.6
PGD	CADETS	ThreaTrace	0.3248	0.3196	3.2
		cnnLog	0.3237	0.3437	3.5
		Kairos	0.3368	0.3581	3.7
		flash	0.3896	0.4066	3
		<i>CCLog</i>	0.4290	0.4255	2.7
	ARENA	ThreaTrace	0.5263	0.4078	2.5
		cnnLog	0.4308	0.4409	2.6
		Kairos	0.5672	0.5932	2.2
		flash	0.5975	0.5582	1.9
		<i>CCLog</i>	0.6061	0.6434	2.2

4.4. RQ3: Robustness of *CCLog*

In this paper, we mainly focus on the effectiveness and performance of attack anomaly detection and attack path reconstruction. Experimental results show that *CCLog* achieves the best average detection and provenance performance across different datasets. Specifically, the detection performance shows an average improvement of 18.66% and outperforms the second-best method by 4.23%, while the provenance performance shows an average improvement of 16.78% and outperforms the second-best method by 0.59%. These results demonstrate that *CCLog* has better generalization ability.

However, towards practical deploy, it is always an important point to take robustness and model security into account. To address potential evasion strategies and adversarial attack methods, we have improved the generalization and robustness of *CCLog* by designing a combined BERT+VAE approach, which has enhanced the model's defense capability against attacks to a certain extent.

We employed two representative adversarial attack methods, FGSM [35] and PGD [36], for validation. As shown in Table 6, all evaluated methods experienced performance degradation under adversarial attacks on both the CADETS and ARENA datasets. Among these, *CCLog* achieved the best overall results, outperforming other methods in most scenarios and metrics, and demonstrating the strongest defensive capability. It is worth noting, however, that the performance of all methods declined significantly compared to their non-attacked baselines, indicating a need for further improvements—such as incorporating adversarial training.

4.5. RQ4: Ablation study of *CCLog*

To validate the effectiveness of designed tasks in *CCLog*, we conducted an ablation study comparing the impact of different components on attack detection performance. The results, shown in Table 7, indicate that each tasks have positive contributes across all metrics.

Besides, to evaluate the model's effectiveness in learning the semantic context of logs, we visualize the BERT model's output using Principal Component Analysis (PCA), as shown in Fig. 9. Before performing the

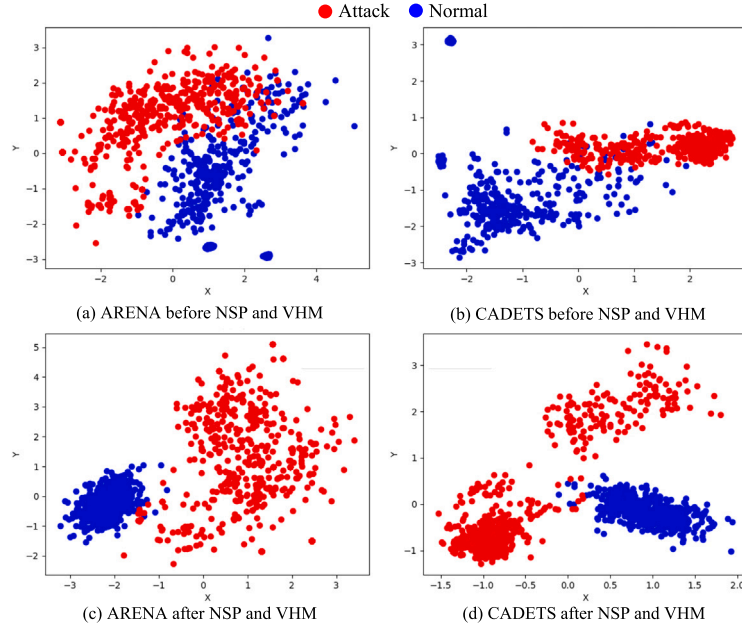


Fig. 9. PCA Visualization of Pre-training Results. (a) and (b) illustrate the visualization of normal and attack node representations after pre-training with only the MLM task, where the feature distributions of both categories remain mixed. In contrast, (c) and (d) depict the distributions after incorporating the VHM and NSP tasks, demonstrating a more distinct separation between normal and attack nodes in the feature space.

Table 7
Ablation result of *CCLog*.

Dataset	Method	F1	Precision@k	AUC-PR	NDCG
ARENA	All task	0.9669	0.8(k = 5)	0.9519	0.8818
	No vhm	0.7240	0.6(k = 5)	0.7488	0.7095
	No nsp	0.8100	0.8(k = 5)	0.8117	0.7528
	No mlm	0.7660	0.6(k = 5)	0.7744	0.7901
	Only vhm	0.3404	0.4(k = 5)	0.4631	0.7962
	Only nsp	0.2593	0.2(k = 5)	0.3627	0.6088
	Only mlm	0.3509	0.4(k = 5)	0.3633	0.7632
CADETS	All task	0.9028	1.0(k = 5)	0.8374	1.0000
	No vhm	0.7899	0.6(k = 5)	0.7012	0.7436
	No nsp	0.8331	0.8(k = 5)	0.6995	0.7382
	No mlm	0.8089	0.8(k = 5)	0.7208	0.6972
	Only vhm	0.5744	0.4(k = 5)	0.3189	0.6867
	Only nsp	0.5332	0.2(k = 5)	0.4251	0.5589
	Only mlm	0.5389	0.2(k = 5)	0.3397	0.5937
clearscope	All task	0.8232	0.8(k = 5)	0.8631	0.9347
	No vhm	0.7094	0.6(k = 5)	0.6893	0.8039
	No nsp	0.7402	0.6(k = 5)	0.7088	0.8391
	No mlm	0.7342	0.8(k = 5)	0.7216	0.8947
	Only vhm	0.5440	0.6(k = 5)	0.5637	0.7883
	Only nsp	0.4988	0.2(k = 5)	0.4374	0.5173
	Only mlm	0.5204	0.4(k = 5)	0.4836	0.6479
THEIA	All task	0.9042	0.8(k = 5)	0.8174	0.7975
	No vhm	0.7983	0.6(k = 5)	0.7012	0.6837
	No nsp	0.7801	0.8(k = 5)	0.7287	0.7236
	No mlm	0.7826	0.6(k = 5)	0.7306	0.7018
	Only vhm	0.6181	0.4(k = 5)	0.5033	0.5774
	Only nsp	0.5510	0.4(k = 5)	0.5872	0.5035
	Only mlm	0.5636	0.4(k = 5)	0.5748	0.5293

VHM and NSP tasks, the feature vectors of normal and attack nodes are intertwined, making them difficult to distinguish. However, after incorporating both VHM and NSP tasks, the feature vectors of normal nodes are well-clustered, and a clear separation between normal and attack nodes emerges in the feature space. This demonstrates that the integration of the three pre-training tasks effectively enhances the BERT model's ability to distinguish normal and attack data in the feature space.

Table 8
Consumption comparison.

Stage	Dataset	Methods	Time (s)	Memory (MB)	CPU
Detection	CADETS	LogBERT	110	10,945	36.5%
		cnolog	133	9830	62.4%
		ThreaTrace	168	13,631	50.9%
		Kairos	83	2359	26.4%
		Flash	99	1901	29.1%
		<i>CCLog (ours)</i>	74	2425	21.9%
	ARENA	LogBERT	103	6685	44.2%
		cnolog	121	7864	72.3%
		ThreaTrace	158	12,517	61.4%
		Kairos	79	1573	33.7%
		Flash	92	1180	30.2%
Provenance	CADETS	<i>CCLog (ours)</i>	67	1769	26.8%
	ARENA	CONAN	102	15,304	25.4%
		MAGIC	179	4132	40.3%
		THREATTRACE	263	7389	68.7%
		UNICORN	324	6227	63.5%
		FLASH	112	6947	56.2%
	ARENA	KAIROS	134	7560	51.8%
		<i>CCLog (ours)</i>	105	3742	37.6%
	ARENA	CONAN	22	8869	18.6%
		MAGIC	95	1526	30.1%
		THREATTRACE	136	3104	55.4%
		UNICORN	114	2798	42.6%
		FLASH	38	3082	33.7%
		KAIROS	62	2973	28.8%
		<i>CCLog (ours)</i>	40	1662	22.3%

4.6. RQ5: The efficiency of *CCLog*

Resource consumption is a critical factor in evaluating the practicality of detection methods. In this study, we focus on two primary aspects of overhead: time consumption and system resource utilization. The performance metrics for detection and provenance stages across datasets are summarized in Table 8, highlighting *CCLog (ours)*'s superior efficiency in terms of time, memory, and CPU utilization.

As shown in Fig. 10, *CCLog* achieves the fastest processing speed in the detection stage on both datasets: 74 s on CADETS and 67 s

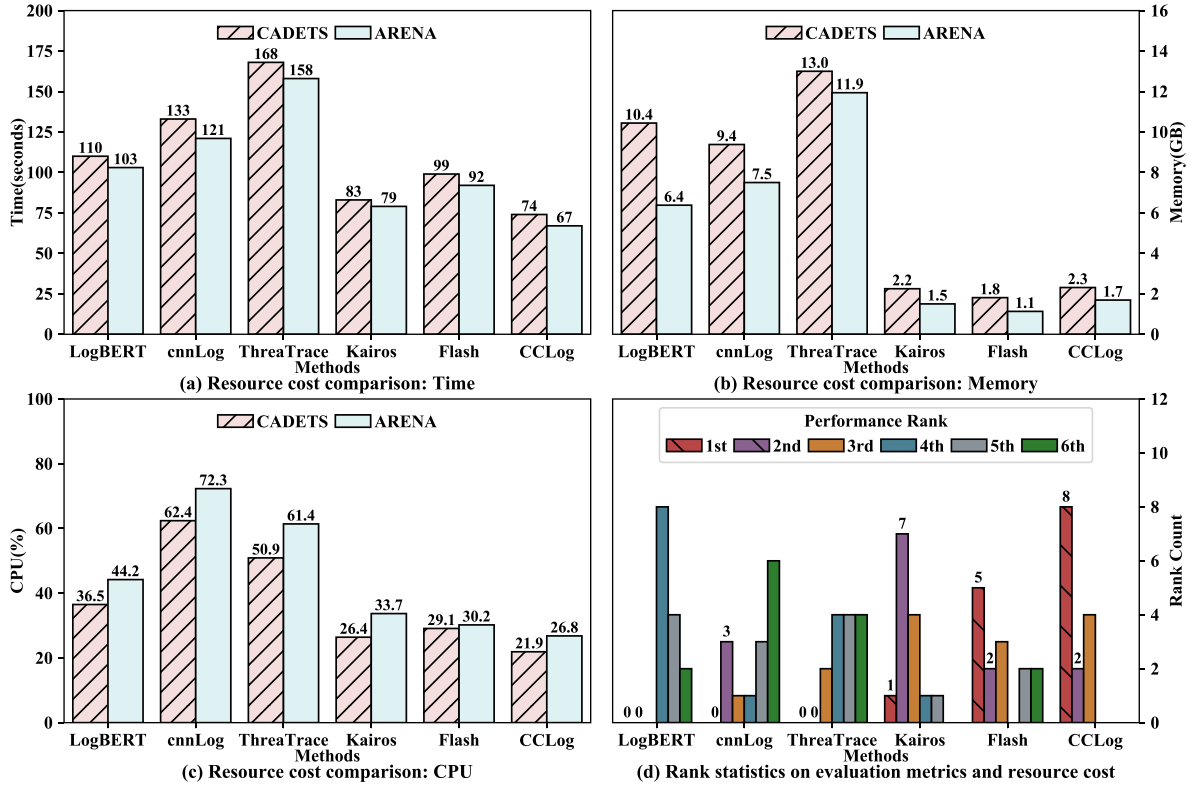


Fig. 10. Resource Consumption Comparison of anomaly entities detection. (a) Time Overhead: Measured by calculating the time required for each anomaly detection method to process the same dataset. (b) and (c) System Resource Overhead: Measured by CPU utilization and memory usage, analyzing the consumption of computing resources by different methods during operation. (d) Rank Statistics: Measured by the ranking of all metrics, indicating that *CCLog* outperforms other methods in the detection phase.

on ARENA, outperforming counterparts like LogBERT (110 s/103 s) and ThreaTrace (168 s/158 s). It also demonstrates low memory consumption (2425 MB on CADETS, 1769 MB on ARENA) and the lowest CPU usage (21.9% on CADETS, 26.8% on ARENA), outperforming even lightweight methods like Flash (99 s/92 s, 1901 MB/1180 MB, 29.1%/30.2%). Rank statistics (Fig. 10(d)) further emphasize *CCLog*'s strengths in the anomaly detection phase. By integrating key performance metrics (Accuracy, Precision, Recall, F1-Score) with resource consumption factors, it achieves a strong overall ranking. Unlike some methods that excel in isolated metrics, *CCLog* effectively balances detection performance and resource efficiency, ensuring high-quality anomaly detection results while minimizing resource demands.

For the provenance stage, as shown in Fig. 11, *CCLog* maintains efficiency: on CADETS, it processes in 105 s with 3742 MB memory and 37.6% CPU, outperforming resource-heavy methods like UNICORN (324 s, 6227 MB, 63.5%) and THREATTRACE (263 s, 7389 MB, 68.7%). On ARENA, it takes 40 s with 1662 MB memory and 22.3% CPU, comparable to FLASH (38 s) but with lower memory than THREATTRACE (3104 MB) and UNICORN (2798 MB), while using less CPU than most alternatives.

Notably, *CCLog*'s efficiency is particularly evident when handling large-scale datasets, such as DARPA, where it completes attack path reconstruction in 105 s. This represents a 50%–70% reduction in time consumption compared to MAGIC, THREATTRACE, and UNICORN. The only method with a slightly lower execution time is CONAN. However, CONAN employs a state-based simple linkage approach for attack path construction, which, while computationally lightweight, suffers from high memory overhead. This is due to its need to store a large number of states, with storage requirements increasing as the number of state snapshots grows. Moreover, from a detection performance perspective,

CCLog significantly outperforms CONAN in terms of accuracy and robustness, making it a more effective solution for practical deployment. Meanwhile, in terms of detection time, *CCLog* performs on par with Flash and Kairos. However, in terms of memory and CPU utilization, *CCLog* is slightly superior to both, which further demonstrates the superiority during the attack path reconstruction phase.

These results confirm *CCLog*'s efficiency advantages, making it suitable for deployment in resource-constrained environments without sacrificing performance.

4.7. RQ6: Key components and params

In *CCLog*, we first compute the attack score using a combined approach that integrates both reconstruction loss and redundancy scoring, as defined in Eq. (7). A node is then identified as anomalous if its score exceeds a predefined **threshold**. So, the **Cluster algorithm** and **Anomaly threshold** are two key components and params of *CCLog*.

4.7.1. Cluster algorithm

To quantify redundancy scoring ($RS(a)$), we apply the K-Means to cluster feature vectors of entities with the same identifier. Compared to other clustering methods such as GMMs and DBSCAN, K-Means is relatively simpler and computationally faster. As shown in Table 9, the time required for each clustering method to complete on the test dataset under different parameter configurations is presented. It can be observed that K-Means demonstrates a significant advantage in processing speed. This makes it more suitable for real-world applications, ensuring high throughput in APT detection and analysis tasks.

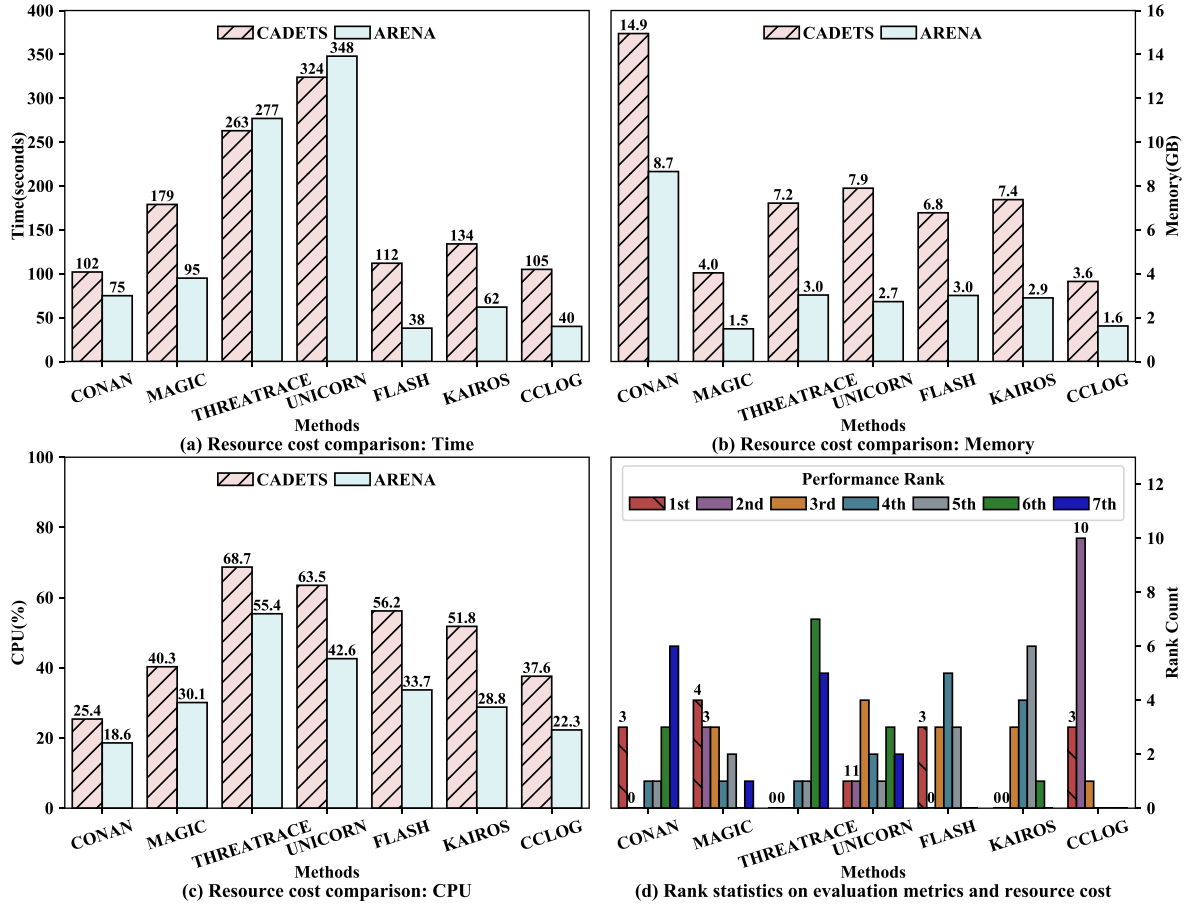


Fig. 11. Resource Consumption Comparison of reconstructing the attack path. (a) Time Overhead: Measured by the time required for each method to reconstruct attack paths when processing the same dataset. (b) and (c) System Resource Overhead: Evaluated based on CPU utilization and memory consumption, analyzing the computational resource demand of each method during execution. (d) Rank Statistics: Integrating key performance metrics to assess the overall ranking of each method. The results highlight that *CCLog* achieves superior overall performance compared to other approaches.

Table 9

Time consumption comparison of different cluster methods (in seconds).

Methods	Params	ARENA	CADETS	clearscope	THEIA	TRACE	Average
K-Means	k = 2	6.27	14.90	16.08	18.93	20.15	15.266
	k = 5	6.46	11.67	17.32	15.79	17.03	13.654
	k = 10	7.15	13.04	18.76	19.52	20.87	15.868
	k = 15	8.89	14.36	20.15	21.07	22.43	17.380
GMMS	k = 2	822.32	828.08	834.05	835.68	837.12	831.450
	k = 5	711.51	716.83	723.17	724.86	726.35	720.544
	k = 10	650.75	656.66	662.49	663.21	664.59	659.540
	k = 15	583.07	588.25	595.83	596.74	598.02	592.382
DBSCAN	eps = 0.1, MinPts = 5	15.63	21.17	26.91	28.45	29.78	24.388
	eps = 0.2, MinPts = 5	13.20	19.02	25.08	26.31	27.56	22.234
	eps = 0.5, MinPts = 5	11.38	16.67	23.09	24.27	25.43	20.168
	eps = 0.9, MinPts = 5	9.80	15.48	21.74	22.59	23.81	18.684
	eps = 0.1, MinPts = 10	16.22	22.19	27.56	28.73	29.91	24.922
	eps = 0.2, MinPts = 10	14.59	19.94	25.93	27.15	28.37	23.196
	eps = 0.5, MinPts = 10	12.11	17.84	23.85	25.08	26.25	21.026
	eps = 0.9, MinPts = 10	8.98	14.39	20.62	22.36	23.59	17.988

4.7.2. Anomaly threshold

In this study, the threshold is set at the 90th percentile of the attack score distribution. Different threshold values lead to varying detection outcomes. Based on experimental validation, *CCLog* achieves optimal performance—as measured by the F1-Score—within the 90th to 95th percentile range across different datasets (See Fig. 12). Therefore, the 90th percentile was selected as the anomaly detection threshold.

4.8. RQ5: Attack case study

We analyze a real APT attack case from the ARENA dataset to illustrate the actual attack path and behavioral chain. As shown in Fig. 13, the attack follows a multi-stage process, where the adversary progressively infiltrates the target system and escalates privileges through multiple steps. For this attack, our method successfully detected 377 attack nodes. The reconstructed attack provenance graph enables rapid identification of critical nodes, reveals the behavioral

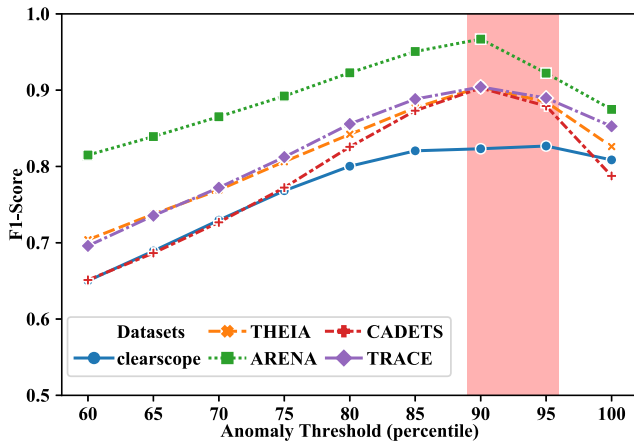


Fig. 12. The F1-Score of anomaly entities detection under different anomaly thresholds. At the 90th percentile, the F1-Score reaches its optimal value on THEIA, CADETS, ARENA, and TRACE datasets. On the clearscope dataset, the F1-Score at the 90th percentile ranks second, slightly lower than that at the 95th percentile.

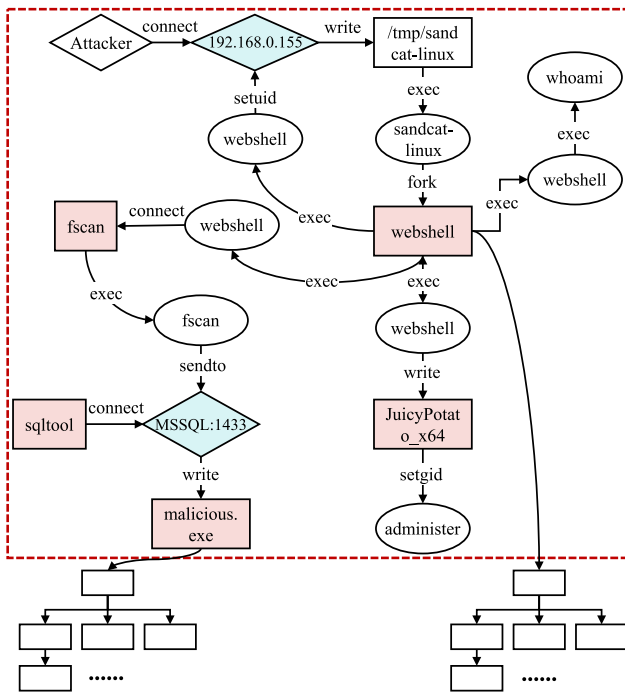


Fig. 13. Illustration of Attack Detection and Provenance Analysis. The attacker exploits a vulnerability to infiltrate the target server and deploys *sandcat-linux* as a remote control backdoor. Once triggered, it spawns a malicious process to execute remote commands (e.g., *ipconfig* for TCP/IP configuration). The attacker then installs a *Webshell* for executing commands like 'whoami' to verify privileges. Next, the attacker runs *fscan* to locate *MSSQLservices*, using weak credentials to gain access. A *malicious.exe* file is uploaded as a persistent backdoor, followed by privilege escalation to administrator level. Finally, the attacker uses *JuicyPotato_x64.exe* to exploit *MS16-075*, obtaining full system control.

patterns of complex attack chains, and supports multi-dimensional analysis by leveraging node attributes.

The attack path generated by *CCLog* provides a comprehensive mapping of the key steps in the attack scenario, effectively supporting forensic analysis and provenance tracking.

5. Conclusions

To address the challenges of high false positive rates, difficult attack path reconstruction, and high resource consumption in APT attack detection, this paper proposes *CCLog*, a context-aware correlation analysis method for APT detection and provenance analysis. *CCLog* extracts entities and operations from large-scale logs to construct a provenance graph and utilizes a BERT-based pre-trained model for log analysis. Anomaly detection is performed using a VAE, while attack path reconstruction is efficiently solved using a Steiner tree approach with shortest-path heuristics and greedy algorithm.

Experimental results demonstrate that *CCLog* achieves a balanced trade-off between detection accuracy and resource efficiency, making it a practical solution for APT detection. In the APT attack detection phase, *CCLog* achieves an average F1-score of 0.9012 across five datasets including CADETS, ARENA, THEIA, Trace, and clearscope, with an average improvement of 18.66% and a 4.23% improvement over the second-ranked method. In the APT attack provenance phase, it achieves an average F1-score of 0.8663, with an average improvement of 16.78% and a 0.59% improvement over the second-ranked method. Additionally, attack path reconstruction achieves optimal or near-optimal detection performance while reducing resource consumption by an average of 30%.

However, since *CCLog* is based on log analysis, it does not incorporate other APT-related data sources, presenting certain limitations. Future research could explore joint analysis of log data, network traffic, and threat intelligence. By leveraging multi-modal learning and cross-modal feature fusion, APT detection capabilities can be enhanced. Besides, potential evasion strategies and adversarial attacks also need to be addressed to improve the robustness and security.

CRedit authorship contribution statement

Zhichao Hu: Writing – review & editing, Writing – original draft, Visualization, Methodology, Investigation, Conceptualization. **Likun Liu:** Writing – review & editing, Software, Project administration, Funding acquisition. **Hongjie Li:** Writing – original draft, Visualization, Validation, Methodology, Data curation, Conceptualization. **Chen Song:** Visualization, Validation, Data curation. **Mengmeng Ge:** Writing – review & editing, Writing – original draft, Validation, Supervision, Investigation. **Qing Guo:** Writing – review & editing, Supervision, Resources. **Lina Ma:** Writing – review & editing, Writing – original draft, Visualization, Resources. **Xiangzhan Yu:** Writing – review & editing, Writing – original draft, Supervision, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by the National Key Research and Development Program of China (No. 2022YFC3321100).

Data availability

Data will be made available on request.

References

- [1] Z. Weng, W. Zhang, T. Zhu, Z. Dou, H. Sun, Z. Ye, Y. Tian, RT-APT: A real-time APT anomaly detection method for large-scale provenance graph, *J. Netw. Comput. Appl.* 233 (2025) 104036.
- [2] J. Ren, R. Geng, Provenance-based APT campaigns detection via masked graph representation learning, *Comput. Secur.* 148 (2025) 104159.
- [3] A. Goyal, G. Wang, A. Bates, R-CAID: Embedding root cause analysis within provenance-based intrusion detection, in: 2024 IEEE Symposium on Security and Privacy, SP, 2024, pp. 3515–3532, <http://dx.doi.org/10.1109/SP54263.2024.00253>.
- [4] L. Wang, X. Shen, W. Li, Z. Li, R. Sekar, H. Liu, Y. Chen, Incorporating gradients to rules: Towards lightweight, adaptive provenance-based intrusion detection, 2024, arXiv preprint arXiv:2404.14720.
- [5] Z. Cheng, Q. Lv, J. Liang, Y. Wang, D. Sun, T. Pasquier, X. Han, Kairos: Practical intrusion detection and investigation using whole-system provenance, in: 2024 IEEE Symposium on Security and Privacy, SP, IEEE, 2024, pp. 3533–3551.
- [6] Z. Jia, Y. Xiong, Y. Nan, Y. Zhang, J. Zhao, M. Wen, {Magic}: Detecting advanced persistent threats via masked graph representation learning, in: 33rd USENIX Security Symposium (USENIX Security 24), 2024, pp. 5197–5214.
- [7] T. Chen, C. Dong, M. Lv, Q. Song, H. Liu, T. Zhu, K. Xu, L. Chen, S. Ji, Y. Fan, APT-kgl: An intelligent APT detection system based on threat knowledge and heterogeneous provenance graph learning, *IEEE Trans. Dependable Secur. Comput.* (2022).
- [8] K.A. Akbar, Y. Wang, G. Ayode, Y. Gao, A. Singhal, L. Khan, B. Thuraisingham, K. Jee, Advanced persistent threat detection using data provenance and metric learning, *IEEE Trans. Dependable Secur. Comput.* 20 (5) (2022) 3957–3969.
- [9] Z. Yu, S. Yang, Z. Li, L. Li, H. Luo, F. Yang, Logms: a multi-stage log anomaly detection method based on multi-source information fusion and probability label estimation, *Front. Phys.* 12 (2024) 1401857.
- [10] M. Landauer, S. Onder, F. Skopik, M. Wurzenberger, Deep learning for anomaly detection in log data: A survey, *Mach. Learn. Appl.* 12 (2023) 100470.
- [11] H. Guo, S. Yuan, X. Wu, Logbert: Log anomaly detection via bert, in: 2021 International Joint Conference on Neural Networks, IJCNN, IEEE, 2021, pp. 1–8.
- [12] S. Lu, X. Wei, Y. Li, L. Wang, Detecting anomaly in big data system logs using convolutional neural network, in: 2018 IEEE 16th Intl Conf on Dependable, Autonomic and Secure Computing, 16th Intl Conf on Pervasive Intelligence and Computing, 4th Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress (DASC/PiCom/DataCom/CyberSciTech), IEEE, 2018, pp. 151–158.
- [13] L. Yan, C. Luo, R. Shao, Discrete log anomaly detection: a novel time-aware graph-based link prediction approach, *Inform. Sci.* 647 (2023) 119576.
- [14] S.M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, V. Venkatakrishnan, Holmes: real-time apt detection through correlation of suspicious information flows, in: 2019 IEEE Symposium on Security and Privacy, SP, IEEE, 2019, pp. 1137–1152.
- [15] J. Yang, Q. Zhang, X. Jiang, S. Chen, F. Yang, Poirot: Causal correlation aided semantic analysis for advanced persistent threat detection, *IEEE Trans. Dependable Secur. Comput.* 19 (5) (2021) 3546–3563.
- [16] T. Zhu, J. Yu, C. Xiong, W. Cheng, Q. Yuan, J. Ying, T. Chen, J. Zhang, M. Lv, Y. Chen, et al., Aptshield: A stable, efficient and real-time apt detection system for linux hosts, *IEEE Trans. Dependable Secur. Comput.* 20 (6) (2023) 5247–5264.
- [17] N. Yan, Y. Wen, L. Chen, Y. Wu, B. Zhang, Z. Wang, D. Meng, DeepPro: Provenance-based APT campaigns detection via GNN, in: 2022 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), IEEE, 2022, pp. 747–758.
- [18] S. Wang, Z. Wang, T. Zhou, H. Sun, X. Yin, D. Han, H. Zhang, X. Shi, J. Yang, Threatrace: Detecting and tracing host-based threats in node level through provenance graph learning, *IEEE Trans. Inf. Forensics Secur.* 17 (2022) 3972–3987.
- [19] N.H.A. Mutalib, A.Q.M. Sabri, A.W.A. Wahab, E.R.M.F. Abdullah, N. Aldahoul, Explainable deep learning approach for advanced persistent threats (APTs) detection in cybersecurity: A review, *Artif. Intell. Rev.* 57 (11) (2024) 297.
- [20] P. Feng, L. Gai, L. Yang, Q. Wang, T. Li, N. Xi, J. Ma, Dawngnn: Documentation augmented windows malware detection using graph neural network, *Comput. Secur.* (2024) 103788.
- [21] H. Ding, J. Zhai, Y. Nan, S. Ma, {Airtag}: Towards automated attack investigation by unsupervised learning with log texts, in: 32nd USENIX Security Symposium (USENIX Security 23), 2023, pp. 373–390.
- [22] M. Ur Rehman, H. Ahmadi, W. Ul Hassan, Flash: A comprehensive approach to intrusion detection via provenance graph representation learning, in: 2024 IEEE Symposium on Security and Privacy, SP, 2024, pp. 3552–3570, <http://dx.doi.org/10.1109/SP54263.2024.00139>.
- [23] S. Benabderrahmane, P. Valtchev, J. Cheney, T. Rahwan, APT-llm: Embedding-based anomaly detection of cyber advanced persistent threats using large language models, 2025, arXiv:2502.09385. URL: <https://arxiv.org/abs/2502.09385>.
- [24] C. Xiong, T. Zhu, W. Dong, L. Ruan, R. Yang, Y. Cheng, Y. Chen, S. Cheng, X. Chen, CONAN: A practical real-time APT detection system with high accuracy and efficiency, *IEEE Trans. Dependable Secur. Comput.* 19 (1) (2020) 551–565.
- [25] X. Han, T. Pasquier, A. Bates, J. Mickens, M. Seltzer, UNICORN: Runtime provenance-based detector for advanced persistent threats, in: 27th Annual Network and Distributed System Security Symposium, NDSS 2020, The Internet Society, 2020.
- [26] Y. Yin, X. He, Y. Liao, APT attack detection method based on traceability graph, *J. Intell. Knowl. Eng. (ISSN: 2959-0620)* 2 (2) (2024) 83.
- [27] H. Liu, Y. Wang, Z. Su, Z. Wang, Y. Pan, R. Lit, Tracegadget: Detecting and tracing network level attack through federal provenance graph, in: ICC 2024-IEEE International Conference on Communications, IEEE, 2024, pp. 2713–2718.
- [28] M. Adnan, D. Bshara, A. Awad, Forensic analysis of APT attacks based on unsupervised machine learning, *Avrupa Bilim. Ve Teknol. Derg.* (49) (2023) 75–82.
- [29] L. Chen, X. Gong, Analysis of APT attack source tracing in industrial internet environment, in: 2022 International Conference on 6G Communications and IoT Technologies (6GloTT), IEEE, 2022, pp. 81–87.
- [30] T. Li, X. Liu, W. Qiao, X. Zhu, Y. Shen, J. Ma, T-trace: Constructing the apts provenance graphs through multiple syslogs correlation, *IEEE Trans. Dependable Secur. Comput.* (2023).
- [31] Q. Wang, H. Yan, C. Zhao, R. Mei, Z. Han, Y. Zhou, APT attribution for malware based on time series shapelets, in: 2022 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), IEEE, 2022, pp. 769–777.
- [32] K.A. D., Transparent computing engagement 3 data release, accessed 21st january 2020, 2020, <https://github.com/darpa-i2o/transparent-computing>.
- [33] S. Li, F. Dong, X. Xiao, H. Wang, F. Shao, Y. Guo, X. Chen, D. Li, NODLINK: An online system for fine-grained APT attack detection and investigation, in: Proceedings 2024 Network and Distributed System Security Symposium, 2024, <http://dx.doi.org/10.14722/ndss.2024.23204>.
- [34] Z. Cheng, Q. Lv, J. Liang, Y. Wang, D. Sun, T. Pasquier, X. Han, Kairos: Practical intrusion detection and investigation using whole-system provenance, in: 2024 IEEE Symposium on Security and Privacy, SP, 2024, pp. 3533–3551, <http://dx.doi.org/10.1109/SP54263.2024.00005>.
- [35] I.J. Goodfellow, J. Shlens, C. Szegedy, Explaining and harnessing adversarial examples, 2015, URL: <https://arxiv.org/abs/1412.6572>. arXiv:1412.6572.
- [36] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu, Towards deep learning models resistant to adversarial attacks, 2019, URL: <https://arxiv.org/abs/1706.06083>. arXiv:1706.06083.